TECHNICAL REFERENCE DOCUMENT · PART D

Implementation Plan

TransitOps — Smart Transport Operations Platform. Tech stack, project structure, REST API design, RBAC, core business logic, and the suggested build order — derived from the PRD business rules and TRD technical plan.

PROJECT	DOCUMENT TYPE	BASED ON	VERSION
TransitOps	**Implementation Plan**	**PRD v1.0 & TRD v1.0**	**1.0**

PART D — IMPLEMENTATION PLAN

Technology Stack & Project Structure

Stack

Layer	Technology
Backend runtime	Node.js
Backend framework	Express.js
Database	MongoDB
ODM	Mongoose
Auth	JWT (JSON Web Token) + bcrypt for password hashing
Frontend	HTML5, CSS3, vanilla JavaScript (fetch API for REST calls)
API style	RESTful JSON API
Environment config	dotenv (`.env`)
Dev tooling	nodemon (dev server), morgan (request logging)

Project Structure

```
transitops/
├── backend/
│   ├── server.js                    # entry point
│   ├── config/
│   │   └── db.js                    # MongoDB connection
│   ├── models/
│   │   ├── User.js
│   │   ├── Vehicle.js
│   │   ├── Driver.js
│   │   ├── Trip.js
│   │   ├── Maintenance.js
│   │   ├── FuelLog.js
│   │   └── Expense.js
│   ├── controllers/
│   │   ├── authController.js
│   │   ├── vehicleController.js
│   │   ├── driverController.js
│   │   ├── tripController.js
│   │   ├── maintenanceController.js
│   │   ├── fuelExpenseController.js
│   │   └── reportController.js
│   ├── routes/
│   │   ├── authRoutes.js
│   │   ├── vehicleRoutes.js
│   │   ├── driverRoutes.js
│   │   ├── tripRoutes.js
│   │   ├── maintenanceRoutes.js
│   │   ├── fuelExpenseRoutes.js
│   │   └── reportRoutes.js
│   ├── middleware/
│   │   ├── authMiddleware.js        # JWT verification
│   │   ├── roleMiddleware.js        # RBAC guard
│   │   └── errorMiddleware.js
│   ├── utils/
│   │   └── csvExport.js
│   ├── .env
│   └── package.json
│
└── frontend/
    ├── index.html                   # login
    ├── dashboard.html
    ├── vehicles.html
    ├── drivers.html
    ├── trips.html
    ├── maintenance.html
    ├── fuel-expense.html
    ├── reports.html
    ├── css/
    │   ├── style.css
    │   └── components.css
    └── js/
        ├── api.js                   # centralized fetch wrapper + auth header
        ├── auth.js
        ├── dashboard.js
        ├── vehicles.js
        ├── drivers.js
        ├── trips.js
```

```
        ├── maintenance.js
        ├── fuelExpense.js
        └── reports.js
```

PART D — IMPLEMENTATION PLAN

REST API Design

RESTful JSON API. Every route except `/api/auth/login` requires a valid JWT; role-restricted routes are additionally gated by `roleMiddleware.js`.

Auth

Method	Route	Access	Description
POST	/api/auth/register	Public (or admin-only)	Create user account
POST	/api/auth/login	Public	Returns JWT + role
GET	/api/auth/me	Authenticated	Current user profile

Vehicles

Method	Route	Access
GET	/api/vehicles	All authenticated (supports `?status=&type=&region=`)
GET	/api/vehicles/:id	All authenticated
POST	/api/vehicles	Fleet Manager
PUT	/api/vehicles/:id	Fleet Manager
DELETE	/api/vehicles/:id	Fleet Manager

Drivers

Method	Route	Access
GET	/api/drivers	All authenticated
GET	/api/drivers/:id	All authenticated
POST	/api/drivers	Safety Officer
PUT	/api/drivers/:id	Safety Officer
DELETE	/api/drivers/:id	Safety Officer

Trips

Method	Route	Access
GET	/api/trips	All authenticated
POST	/api/trips	Dispatcher (creates as `draft`)
PUT	/api/trips/:id	Dispatcher (edit while `draft`)
POST	/api/trips/:id/dispatch	Dispatcher — runs BR-1..BR-6 validation
POST	/api/trips/:id/complete	Dispatcher — runs BR-7
POST	/api/trips/:id/cancel	Dispatcher — runs BR-8

REST API Design (continued)

Maintenance

Method	Route	Access
GET	/api/maintenance	Fleet Manager, Financial Analyst
POST	/api/maintenance	Fleet Manager — triggers BR-9
POST	/api/maintenance/:id/close	Fleet Manager — triggers BR-10

Fuel Logs & Expenses

Method	Route	Access
GET / POST	/api/fuel-logs	Financial Analyst
GET / POST	/api/expenses	Financial Analyst

Dashboard & Reports

Method	Route	Description
GET	/api/dashboard/kpis	Returns all KPI values
GET	/api/reports/fuel-efficiency	Per-trip / per-vehicle efficiency
GET	/api/reports/operational-cost	Per-vehicle cost roll-up
GET	/api/reports/roi	Per-vehicle ROI
GET	/api/reports/export/csv?type=	Streams CSV for the requested report type

RBAC Permission Matrix

Role	Permissions
fleet_manager	Full CRUD: Vehicle, Maintenance; Read: Trip, Fuel/Expense
dispatcher	Full CRUD: Trip; Read-only: Vehicle, Driver
safety_officer	Full CRUD: Driver; Read-only: Trip
financial_analyst	Full CRUD: Fuel Log, Expense; Read-only: everything (for reports)

PART D — IMPLEMENTATION PLAN

Core Business Logic — Implementation Notes

Trip lifecycle transitions (`controllers/tripController.js`) — the most business-rule-dense code in the app.

```
// controllers/tripController.js

exports.dispatchTrip = async (req, res, next) => {
  try {
    const trip = await Trip.findById(req.params.id).populate('vehicle driver');
    if (!trip) return res.status(404).json({ message: 'Trip not found' });

    const { vehicle, driver } = trip;

    if (vehicle.status !== 'available') {
      return res.status(400).json({ message: 'Vehicle is not available.' }); // BR-2, BR-4
    }
    if (driver.status !== 'available') {
      return res.status(400).json({ message: 'Driver is not available.' }); // BR-4
    }
    if (driver.status === 'suspended' || driver.licenseExpiryDate < new Date()) {
      return res.status(400).json({ message: 'Driver is suspended or license expired.' }); // BR-3
    }
    if (trip.cargoWeight > vehicle.maxLoadCapacity) {
      return res.status(400).json({ message: 'Cargo weight exceeds vehicle capacity.' }); // BR-5
    }

    vehicle.status = 'on_trip';
    driver.status = 'on_trip';
    trip.status = 'dispatched';

    await Promise.all([vehicle.save(), driver.save(), trip.save()]); // BR-6

    res.json(trip);
  } catch (err) { next(err); }
};
```

```
// completeTrip() — BR-7: releases vehicle + driver back to available
trip.actualDistance = actualDistance;
trip.fuelConsumed = fuelConsumed;
trip.status = 'completed';
trip.vehicle.odometer += actualDistance;
trip.vehicle.status = 'available';
trip.driver.status = 'available';
await Promise.all([trip.vehicle.save(), trip.driver.save(), trip.save()]);

// cancelTrip() — BR-8: only releases if it was already dispatched
if (trip.status === 'dispatched') {
  trip.vehicle.status = 'available';
  trip.driver.status = 'available';
  await Promise.all([trip.vehicle.save(), trip.driver.save()]);
}
trip.status = 'cancelled';
await trip.save();
```

> Use a MongoDB session/transaction in production for the multi-document saves above — the TRD snippet uses `Promise.all()` for simplicity, which is "atomic-ish" but not a true transaction.

Maintenance Logic & Non-Functional Requirements

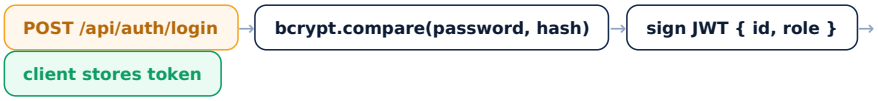
```
// controllers/maintenanceController.js

exports.createMaintenance = async (req, res, next) => {
  const maintenance = await Maintenance.create({ ...req.body, status: 'open' });
  await Vehicle.findByIdAndUpdate(maintenance.vehicle, { status: 'in_shop' }); // BR-9
  res.status(201).json(maintenance);
};

exports.closeMaintenance = async (req, res, next) => {
  const maintenance = await Maintenance.findById(req.params.id).populate('vehicle');
  maintenance.status = 'closed';
  maintenance.dateEnd = new Date();
  await maintenance.save();

  if (maintenance.vehicle.status !== 'retired') {
    maintenance.vehicle.status = 'available'; // BR-10
    await maintenance.vehicle.save();
  }
  res.json(maintenance);
};
```

Authentication Flow



`authMiddleware.js` verifies the JWT on every protected route and attaches `req.user`. `roleMiddleware.js` exposes `requireRole('fleet_manager', 'dispatcher')` — rejects with 403 if `req.user.role` isn't in the allowed list.

Non-Functional Requirements

Concern	Requirement
Validation	Both client-side (UX feedback) and server-side (source of truth) — never rely on frontend validation alone for business rules
Error handling	Centralized Express error-handling middleware (`errorMiddleware.js`) returning a consistent `{ message, status }` JSON shape
Environment config	`.env` for `MONGO_URI`, `JWT_SECRET`, `PORT` — never hardcode secrets
Password security	bcrypt with salt rounds ≥ 10; never store or log plaintext passwords
CORS	Enable `cors` middleware scoped to the frontend's origin
Logging	`morgan` for request logs in development

PART D — IMPLEMENTATION PLAN

Suggested Build Order

A 10-step incremental path — each step produces a working, testable slice of the app.

1. Project scaffold (`backend/` + `frontend/`), MongoDB connection, `.env` setup
2. `User` model + auth (register/login/JWT) + `authMiddleware` + `roleMiddleware`
3. `Vehicle` and `Driver` models + CRUD routes/controllers + basic frontend pages
4. `Trip` model + full lifecycle logic (BR-1 to BR-8)
5. `Maintenance` model + BR-9/BR-10 sync logic
6. `FuelLog` + `Expense` models + operational cost aggregation
7. Dashboard KPIs endpoint + frontend dashboard page
8. Reports endpoints (Fuel Efficiency, Utilization, ROI) + CSV export
9. RBAC polish across all routes + run through the demo scenario end-to-end
10. Styling pass, responsiveness check, final bug fixes

Demo Scenario (End-to-End Validation Script)

1. Register vehicle `Van-05`, capacity 500 kg → status `available`
2. Register driver `Alex`, valid license
3. Create trip, cargo weight 450 kg (as `draft`)
4. Attempt dispatch with 550 kg cargo first → API returns `400` (BR-5 blocked), then correct to 450 kg → dispatch succeeds
5. Confirm Vehicle + Driver auto-flip to `on_trip` in the database
6. Complete trip (enter odometer + fuel consumed) → both auto-flip to `available`
7. Create maintenance record (Oil Change) → vehicle auto-flips to `in_shop`, disappears from a new trip's vehicle dropdown
8. Close maintenance → vehicle back to `available`
9. Check Reports: Fuel Efficiency and Operational Cost reflect the above